

VEHIQ: Technical Feasibility & Sustainability Analysis

Software & Hardware Requirements

1. TECHNICAL FEASIBILITY SCORE: 7/10 ✓ VIABLE

Summary

VEHIQ sensor integration is **technically feasible** for MVP/Beta phases. Core OBD-II protocol is mature and well-documented. Main challenges are in error handling, data persistence, and accuracy validation—not fundamental technology gaps.

2. HARDWARE REQUIREMENTS & FEASIBILITY

2.1 OBD-II Dongle (Primary Hardware)

ELM327 Bluetooth Dongle

Specification	Details	Feasibility	Cost
Type	Bluetooth v2.0/4.0 wireless adapter	✓ Available	₹500–1500
Connector	J1962 OBD-II standard port	✓ Universal	Included
Protocol Support	CAN, ISO 9141, K-LINE, VPW	✓ Sufficient	Native
Latency	500ms–2s per PID query	⚠ Acceptable	—
Range	10m Bluetooth range	✓ Adequate	—
Power	12V from vehicle (via OBD port)	✓ Auto-powered	—
Compatibility	95%+ vehicles post-1996	✓ Wide support	—

Feasibility: 9/10 ✓

- Mature product, thousands of clones available on Amazon India
- Low cost makes consumer adoption feasible
- No manufacturing needed; off-the-shelf hardware

Concerns:

- ⚠ Clone quality variable; failure rates 15–20%
- ⚠ Bluetooth pairing reliability 70–80% first attempt
- ⚠ No standardized firmware; incompatibilities exist

Mitigation:

- ✓ Recommend branded ELM327 (not clones) in MVP
 - ✓ Implement retry logic for failed connections
 - ✓ Provide user setup guide with troubleshooting
 - ✓ Support premium dongle option (₹3000–5000 for commercial)
-

2.2 Alternative Hardware Options

Option	Cost	Latency	Reliability	Production Use
ELM327 Bluetooth	₹500–1500	500ms–2s	80%	✓ MVP
Direct CAN Bus Adapter	₹200–500	<100ms	95%	✓ Commercial fleets
Native Vehicle APIs	\$50–200/yr	100–500ms	99%	✓ Phase 3 (OEM)
Telematics Provider	₹5–20/mo	1–5s	98%	✓ Insurance/Fleet

Recommendation: Start with ELM327 for MVP; plan CAN bus adapter for commercial customers in Phase 2.

2.3 Vehicle Sensor Types (Hardware-Level)

Standard PIDs (Mandatory)

Sensor	Hardware	PID	Unit	Normal Range	Feasibility
Battery Voltage	ECU + sensor	0x01	V	12.0–14.5	✓ 100%
Engine RPM	ECU	0x0C	RPM	600–3000	✓ 100%
Coolant Temp	Thermistor	0x05	°C	80–100	✓ 100%
Throttle Position	Potentiometer	0x11	%	0–100	✓ 95%
Fuel Level	Float sensor	0x2F	%	0–100	✓ 90%

Oil Pressure	Pressure sensor	0x5D	PSI	25–65	✓ 85%

Extended Sensors (Via CAN Bus)

Sensor	Hardware	Availability	Feasibility	Note
Tire Pressure (4x)	TPMS module	✓ Post-2006	✓ 90%	Varies by vehicle
Tire Temp (4x)	Thermistor	⚠ Premium vehicles	✓ 60%	Luxury/sports cars only
Brake Pad Wear	Hall sensor	⚠ Premium vehicles	✓ 70%	Aftermarket available
Transmission Temp	Thermistor	⚠ Post-2010	✓ 75%	Automatic only
ABS Status	ECU flag	✓ Post-2000	✓ 95%	Standard in most markets
O ₂ Sensor	Oxygen sensor	✓ Pre-2000+	✓ 99%	Multiple per vehicle

Feasibility: 8/10 ✓

- Standard PIDs work on 95%+ of vehicles
- Extended sensors have regional/model variations
- Mock data sufficient for MVP; real data in Phase 2

3. SOFTWARE REQUIREMENTS & FEASIBILITY

3.1 Backend Architecture

Tech Stack

Component	Technology	Version	Feasibility	Notes
Framework	FastAPI	0.104+	✓ 10/10	Async-ready, built-in async
OBD Library	python-obd or pyobd	0.0.10+	⚠ 7/10	Unmaintained since 2020; needs fork
Database	PostgreSQL	14+	✓ 9/10	Production-ready
Time-Series DB	InfluxDB	2.7+	✓ 9/10	Perfect for sensor data
Cache	Redis	7+	✓ 8/10	Optional (nice-to-have)
Message Queue	Celery + RabbitMQ	Latest	✓ 8/10	For async OBD polling
API Gateway	FastAPI + CORS	Latest	✓ 9/10	Built-in
Auth	JWT + OAuth2	Standard	✓ 9/10	FastAPI native
Logging	Sentry + structlog	Latest	✓ 9/10	Error tracking
Deployment	Docker + Kubernetes	Latest	✓ 9/10	Container-ready

Overall Feasibility: 8/10 ✓

3.2 Core Modules (Software)

Module 1: OBD-II Handler

Purpose: Connect to vehicle, poll PIDs, handle errors

File: backend/models/obd.py

Lines of Code: ~500-800

Development Time: 1-2 weeks

Feasibility: 8/10 ✓

Requirements:

- ✓ Bluetooth connection management
- ✓ PID query with retry logic (3x retries, 2s timeout)
- ✓ CRC error checking
- ✓ Mock data generator (for MVP)
- ✓ Connection pooling (for concurrent requests)
- ⚠ ELM327-specific firmware detection (optional)

Critical Features Missing (from docs):

- ✗ Timeout handling for hung connections
- ✗ Partial read recovery (read 15/20 sensors, 5 fail)
- ✗ Fallback to available PIDs (not supported → skip gracefully)
- ✗ Vehicle auto-detection (auto-query make/model from VIN)

Risk Level: MEDIUM (implement error handling first)

Module 2: Sensor Data Store

Purpose: Persist sensor readings (JSON → DB)

File: backend/models/sensors.py

Lines of Code: ~300-400

Development Time: 1 week

Feasibility: 7/10 ⚠

MVP Implementation (JSON):

- Simple file-based logging (reading_20240815_10:30.json)
- No schema versioning
- Problem: Unscalable beyond 10K readings

Production Implementation (PostgreSQL):

Schema needed:

```
CREATE TABLE sensor_readings (  
  id SERIAL PRIMARY KEY,  
  vehicle_id VARCHAR(50) NOT NULL,  
  sensor_type VARCHAR(50) NOT NULL, -- "battery_voltage"  
  value FLOAT NOT NULL,  
  unit VARCHAR(10) NOT NULL,  
  timestamp TIMESTAMP NOT NULL,  
  anomaly_flag BOOLEAN DEFAULT FALSE,  
  severity VARCHAR(20), -- "low", "medium", "high", "critical"  
  UNIQUE(vehicle_id, sensor_type, timestamp)  
);  
  
CREATE TABLE vehicle_profiles (  
  vehicle_id VARCHAR(50) PRIMARY KEY,  
  make VARCHAR(50),  
  model VARCHAR(50),  
  year INT,  
  engine_type VARCHAR(20),  
  mileage INT,  
  created_at TIMESTAMP DEFAULT NOW()  
);
```

Issues:

- ✗ No schema migration plan (MVP JSON → Production PostgreSQL)
- ✗ No retention policy automated
- ✗ No archival strategy for old data
- ✗ InfluxDB schema undefined (time-series format)

Risk Level: MEDIUM (requires careful migration)

Module 3: Anomaly Detection

Purpose: Compare sensor readings vs. thresholds

File: backend/models/anomaly.py

Lines of Code: ~400-600

Development Time: 1-2 weeks

Feasibility: 5/10 ⚠ ⚠

Current Approach (from docs):

- Static min/max thresholds per sensor
- Problem: Ignores context (temperature, driving mode, et

Issues:

- ✗ No context-aware thresholds
 - Battery 11.8V at 5°C = normal (cold weather)
 - Battery 11.8V at 30°C = failure risk
 - No formula provided for seasonal adjustment
- ✗ No sensor-specific logic
 - Tire pressure: 28 PSI in winter $\neq$ 28 PSI in summer (+
 - Oil pressure: Depends on RPM (300 PSI at 5000 RPM $\neq$ 3
 - Engine temp: Depends on ambient + load (open-loop vs.
- ✗ No historical trend detection
 - Gradual battery degradation: 14.2V $\rightarrow$ 13.8V $\rightarrow$ 13.1V $\rightarrow$
 - Current approach: Only flags 11.9V reading, misses de

Implementation Required:

- ✓ Temperature-aware thresholds
- ✓ RPM-dependent thresholds
- ✓ Historical trend analysis (7-day rolling average)
- ✓ Contextual anomaly scoring (Isolation Forest or Z-scor
- ✓ False positive filtering

Risk Level: HIGH (accuracy unvalidated)

Module 4: Sensor Fusion Engine

Purpose: Merge photo + sensor data into single diagnosis

File: backend/models/fusion.py

Lines of Code: ~300-500

Development Time: 1-2 weeks

Feasibility: 6/10 $\triangle$

Current Rule (from docs):

- If photo + sensors agree → boost confidence by 0.3
- Problem: No weighting, no confidence formula

Issues:

✗ No weighted confidence calculation

Photo (60% confidence) + Sensor (95% confidence)

→ Current: $0.6 + 0.3 = 0.9?$ (undefined)

→ Should be: $\text{weighted_avg} = 0.4 \times 0.6 + 0.6 \times 0.95 = 0.93$

✗ Conflict handling undefined

Photo says "brake pad wear" (80% confidence)

Sensors say "battery low" (95% confidence)

→ "Prefer sensor data, lower confidence" – but how much?

✗ DTC mapping incomplete

- Only 5 P-codes mapped (P0420, P0171, P0300, P0505, P0101)

- Real vehicles throw 200+ codes

- Need OBD DTC database integration

Implementation Required:

✓ Bayesian fusion: $P(\text{diagnosis} \mid \text{photo}, \text{sensor}) = P(\text{photo} \mid \text{diagnosis}) \times P(\text{sensor} \mid \text{diagnosis}) \times P(\text{diagnosis})$

✓ Weighted confidence: $w_{\text{photo}} \times \text{conf_photo} + w_{\text{sensor}} \times \text{conf_sensor}$

✓ Conflict resolution: Flag for manual review if sources disagree

✓ DTC database: JSON lookup table or API to obd-codes.com

Risk Level: MEDIUM-HIGH (validation needed on real data)

Module 5: Diagnostic Trouble Code (DTC) Handler

Purpose: Read & interpret OBD-II error codes

File: backend/models/dtc.py

Lines of Code: ~200-300

Development Time: 3-5 days

Feasibility: 6/10 ▲

Current Status:

- 5 codes documented (P0420, P0171, P0300, P0505, P0101)

- Problem: SAE defines 200+ standard P-codes; OEMs add proprietary

Implementation Required:

DTC Database Schema:

```
{
  "P0420": {
    "title": "Catalyst System Efficiency Below Threshold",
    "description": "Oxygen sensor signal indicates...",
    "severity": "medium",
    "part": "catalytic_converter",
    "probable_causes": ["bad_o2_sensor", "exhaust_leak",
    "estimated_cost": "₹15000-25000"
  },
  ...200+ more codes
}
```

Missing:

- ✗ OEM-specific codes (Toyota, Honda, Ford, Maruti varian
- ✗ Severity auto-classification
- ✗ Part mapping (P0420 → catalytic converter)
- ✗ Cost estimation per region (India vs. US vs. EU)

Data Source Options:

1. Static JSON database (500 codes) – ✓ MVP
2. API integration (obd-codes.com) – ⚠ Rate limits
3. Offline SQLite database – ✓ Production

Risk Level: LOW (database built incrementally)

3.3 Frontend Requirements

Tech Stack

Component	Technology	Version	Feasibility
Framework	Next.js	14+	✓ 10/10
Styling	Tailwind CSS	3+	✓ 10/10

State	React Context	Latest	✓ 9/10
Charts	Recharts	2.10+	✓ 9/10
HTTP Client	Axios or Fetch	Latest	✓ 10/10
Camera (mobile)	React-Camera-Pro	Latest	✓ 8/10

Overall Feasibility: 9/10 ✓

Pages & Components Required

Page/Component	Purpose	Status	LOC	Dev Time	Feasibility
/ (Home)	Navigation hub	✓ Exists	200	2 days	10/10
/result	Photo diagnosis display	✓ Exists	400	3 days	10/10
/sensor-diagnostics	OBD-II scan interface	⚠ Partial	600	1 week	8/10
SensorDashboard	Live sensor display	⚠ Partial	400	5 days	8/10
DiagnosisCard	Result summary	✓ Exists	200	2 days	10/10
PriceComparison	Parts/ costs	✓ Exists	300	3 days	10/10
ChatWidget	Floating chatbot	✓ Exists	500	5 days	9/10
VehicleProfileForm	VIN/make/ model input	⦿ Missing	300	3 days	9/10

AnomalyAlerts	Severity badges	Missing	150	2 days	9/10
SensorChart	7-day trend plot	Missing	250	3 days	8/10

Feasibility: 8/10 ✓ (Most components straightforward; data flow needs clarification)

4. CRITICAL SOFTWARE GAPS

Gap 1: Error Handling & Resilience

- ✗ NO RETRY LOGIC
 - OBD-II reads fail 5-10% of time (Bluetooth interference)
 - Current: Single attempt → returns null
 - Required: Implement exponential backoff (0.5s, 1s, 2s)
- ✗ NO TIMEOUT MANAGEMENT
 - ELM327 can hang for 30+ seconds on bad commands
 - Current: No timeout specified
 - Required: Set 2-5s timeout per query; kill connection if no response
- ✗ NO PARTIAL READ HANDLING
 - Read 30 sensors, 5 fail
 - Current: Return all-or-nothing
 - Required: Return available data, flag missing sensors
- ✗ NO CONNECTION POOLING
 - MVP: Single connection per request
 - Problem: 100 concurrent users → 100 connections
 - Required: Queue requests, max 5 concurrent connections

```
Code Needed:
def query_with_retry(pid, max_retries=3, timeout=2):
```

```

for attempt in range(max_retries):
    try:
        response = connection.query(pid, timeout=time
        return response
    except TimeoutError:
        if attempt == max_retries - 1:
            return None # Mark as unavailable
            sleep(0.5 * (2 ** attempt)) # Exponential ba
return None

```

Feasibility to Fix: ✓ LOW (1-2 days)

Priority: ● CRITICAL (must fix before MVP)

Gap 2: Data Persistence Strategy

✗ JSON STORAGE (MVP)

Files: sensor_readings_20240815.json

Problem:

- No schema versioning
- No atomic writes (data corruption on crash)
- No query capability (can't search "all readings for veh
- Scales to 10K readings max (~2 MB file size)

✗ DATABASE MIGRATION PATH UNDEFINED

- MVP writes JSON
- Phase 2 should write PostgreSQL
- Question: How do you migrate 100K JSON records to datab
- Current answer: "We'll figure it out in Phase 2" ← DANG

Required Implementation:

MVP (Week 1):

```

CREATE TABLE sensor_readings (
    id SERIAL PRIMARY KEY,
    vehicle_id VARCHAR,
    sensor_type VARCHAR,
    value FLOAT,
    unit VARCHAR,
    timestamp TIMESTAMP UNIQUE,

```

```
anomaly BOOLEAN,  
severity VARCHAR  
);
```

Migration Script (Phase 2):

for record in json_files:

```
    INSERT INTO sensor_readings (...) VALUES (...)
```

Feasibility to Fix: ✓ MEDIUM (1 week for full implementation)

Priority: ● HIGH (needed by Phase 2)

Gap 3: Accuracy & Validation

✗ UNVALIDATED THRESHOLDS

- Docs provide thresholds for 10 sensors
- Real vehicles have 30+ sensors with different normal ranges
- No A/B testing strategy

✗ NO GROUND TRUTH

- How do you know if diagnosis is correct?
- Plan: Compare with professional mechanic scan (manual validation)
- Timeline: Phase 2 (8 weeks)

✗ FALSE POSITIVE / NEGATIVE RATES UNKNOWN

- Target: <5% false positive, <10% false negative
- Current: No metrics

Required:

- Collect 100 vehicle scans (real + mechanic validation)
- Measure accuracy per sensor type
- Tune thresholds per make/model
- Calculate confidence intervals

Feasibility: △ MEDIUM (data-science heavy, 6-8 weeks)

Priority: ● CRITICAL (liability risk if not validated)

Gap 4: Library Maintenance

- ✗ PYOBD UNMAINTAINED SINCE 2020
 - Last commit: ~4 years ago
 - Risk: New vehicle protocols emerge, library breaks
 - Solution: Fork library internally or migrate to python-

Maintenance Plan:

Phase 2: Fork pyobd → vehiq-obd

- Add error handling
- Add logging
- Pin to Python 3.9+

Phase 3: Migrate to python-can library (more actively mai

Feasibility: ✓ MEDIUM (1-2 weeks for fork)

Priority: ● MEDIUM (needed before production)

5. DEPLOYMENT & INFRASTRUCTURE REQUIREMENTS

5.1 Backend Deployment

Environment	Platform	Cost/ Month	Uptime	Feasibility
MVP	Render.com	₹1,500	95%	✓ 9/10
Beta	Render.com (upgraded)	₹2,500	98%	✓ 9/10
Production	AWS EC2 + RDS	₹10,000	99.9%	✓ 9/10

Render.com Limitations:

- Free tier: Dyno sleeps after 15 min inactivity (bad for OBD polling)
- Production tier: ₹1,500/month (acceptable)
- Scaling: Vertical scaling only (no auto-scale)

AWS Alternative:

- EC2 t3.medium: ₹1,500/month
- RDS PostgreSQL: ₹2,000/month
- InfluxDB Cloud: ₹1,000/month
- Total: ₹4,500/month (but includes auto-scaling, 99.9% SLA)

Recommendation: Render for MVP; AWS for production.

5.2 Frontend Deployment

Platform	Cost	Deployment Time	Feasibility
Vercel (Next.js native)	₹0–2,000 (free tier sufficient MVP)	1 minute	✓ 10/10
Netlify	₹0–1,500	1 minute	✓ 10/10
AWS S3 + CloudFront	₹500–2,000	5 minutes	✓ 9/10

Recommendation: Vercel (free tier) for MVP; upgrade to Pro (₹2,000/month) at 10K+ users.

5.3 Database Deployment

Database	MVP	Beta	Production	Cost
PostgreSQL	SQLite (local)	Render PostgreSQL	AWS RDS	₹500–2,000/mo
InfluxDB	None	InfluxDB Cloud	InfluxDB Cloud	₹0–1,000/mo

Redis	Optional	Optional	Render Redis	₹500/mo
-------	----------	----------	--------------	---------

Recommended Stack:

```
MVP (Week 1):  
- PostgreSQL: Local SQLite (during dev)  
- InfluxDB: None (not needed yet)  
- Redis: None  
  
Beta (Month 2):  
- PostgreSQL: Render PostgreSQL ($10/mo)  
- InfluxDB: InfluxDB Cloud ($100/mo)  
- Redis: Render Redis ($10/mo)  
  
Production (Month 4+):  
- PostgreSQL: AWS RDS ($500/mo)  
- InfluxDB: InfluxDB Cloud ($500/mo)  
- Redis: ElastiCache ($200/mo)
```

6. SUSTAINABILITY ANALYSIS

6.1 Operational Sustainability

Metric	MVP	Beta	Production	Trend
Infrastructure Cost	₹2,100	₹5,000	₹25,000	 Linear growth
Team Cost	₹50,000	₹100,000	₹200,000	 2x per phase
Total Monthly Burn	₹52,100	₹105,000	₹225,000	 Unsustainable

Revenue	₹0	₹0	₹0 (required: ₹225K)	⦿ CRITICAL
Runway	2–3 months	1–2 months	Not viable	⦿ CRISIS

Verdict: ⦿ NOT SUSTAINABLE without revenue model

6.2 Technical Debt

Debt Item	Accumulation	Phase to Fix	Cost to Fix
Error handling	Grows weekly	Phase 2	₹50,000 (2 weeks)
Testing coverage	Grows monthly	Phase 3	₹100,000 (4 weeks)
Documentation	Static (not maintained)	Phase 2	₹20,000 (1 week)
Library maintenance	Accumulates as versions change	Phase 3	₹50,000 (fork)
Threshold tuning	New per vehicle make/model	Phase 2 onwards	₹20,000/quarter
DTC database	Grows 50+ codes/year	Phase 2 onwards	₹10,000/quarter

Total Tech Debt: ₹250,000+ if not managed

6.3 Maintenance Requirements (Monthly)

Task	Time/ Month	Feasibility	Automation?
Monitoring & alerts	8 hours	✓	Partial (DataDog)

Bug fixes	20 hours	✓	No
DTC database updates	4 hours	✓	Yes (script)
Threshold tuning	8 hours	⚠	No (data-science)
Security updates	4 hours	✓	Partial (Dependabot)
User support	16 hours	⚠	No (hardware issues)
Total	60 hours	~1 FTE	~50%

Verdict: Requires dedicated DevOps engineer (₹30,000–50,000/month)

7. TECHNOLOGY MATURITY MATRIX

Mature (Safe to Use)

Technology	Maturity	Recommendation
FastAPI	✓ Production-ready (4+ years)	Use in MVP
PostgreSQL	✓ Enterprise-grade (20+ years)	Use in MVP
Next.js	✓ Stable (3+ years)	Use in MVP
Docker	✓ Standard industry tool	Use in MVP
OBD-II protocol	✓ 28-year-old standard	Use in MVP

Emerging (Needs Caution)

Technology	Maturity	Recommendation
InfluxDB	⚠ 2.x has breaking changes	Use in Phase 2 only
Kubernetes	⚠ Overkill for MVP	Wait until 50K+ users
GraphQL	⚠ Adds complexity	Stick with REST API
ML (predictive maintenance)	⚠ Requires 1000+ data points	Phase 4+

At-Risk (Avoid)

Technology	Maturity	Status
pyobd library	🕒 Unmaintained (4 years)	Plan fork or migration
Custom CAN bus parser	🕒 Complex, error-prone	Use existing libraries

8. GO/NO-GO TECHNICAL ASSESSMENT

MVP (3–4 Weeks)

Requirement	Status	Risk	Action
OBD-II hardware available	✓ YES	LOW	Finalize dongle selection
FastAPI backend viable	✓ YES	LOW	Start development

Next.js frontend ready	✓ YES	LOW	Start development
	✗ NO	⦿ HIGH	Write spec NOW (2 days)
Error handling spec	✗ NO	⦿ HIGH	Legal review (1 week)
Privacy policy draft	✗ NO	⦿ HIGH	Legal review (1 week)
Mock data generator	⚠ PARTIAL	MEDIUM	Enhance with noise (3 days)
Testing framework	✗ NO	MEDIUM	pytest + conftest (2 days)

MVP Feasibility: 7/10 ⚠ (Technology OK; compliance missing)

Beta (8 Weeks)

Requirement	Status	Risk	Action
Real ELM327 integration	✓ READY	MEDIUM	Integrate (1 week)
100+ vehicle validation	✗ NO	⦿ HIGH	Collect & analyze (6 weeks)
PostgreSQL migration	⚠ PARTIAL	MEDIUM	Complete schema & migration script (1 week)
Compliance audit	✗ NO	⦿ HIGH	GDPR/India DPA (3 weeks)
Load testing (1K users)	✗ NO	MEDIUM	Set up test environment (1 week)
Monitoring (DataDog)	✗ NO	MEDIUM	Integrate + alerting (3 days)

Beta Feasibility: 6/10 ⚠ ⚠ (Data validation is blocker)

Production (4+ Months)

Requirement	Status	Risk	Action
Native vehicle APIs	✗ NOT STARTED	🔴 CRITICAL	Defer to Phase 4
Revenue model working	✗ NO	🔴 CRITICAL	Define NOW (1 week)
99.9% uptime SLA	✗ NO	🔴 CRITICAL	AWS migration (2 weeks)
Regulatory approval	✗ NO	🔴 CRITICAL	Compliance done in Phase 2
Accuracy >95%	✗ UNKNOWN	🟡 HIGH	Validate in Phase 2

Production Feasibility: 4/10 ✗ (Too many blockers)

9. DEVELOPMENT TIMELINE ESTIMATE

MVP (Weeks 1–4)

Task	Owner	Days	Status
OBD-II handler + error handling	Backend	10	🔴 Not started
Sensor data models	Backend	5	🔴 Not started
Anomaly detection basic	Backend	10	🔴 Not started
Sensor fusion engine	Backend	8	🔴 Not started
/sensor-diagnostics page	Frontend	8	🟡 Partial
SensorDashboard component	Frontend	6	🟡 Partial
API integration	Frontend	5	🟡 Partial

Testing + bug fixes	QA	5	🕒 Not started
Privacy policy + Legal	Legal	5	🕒 Not started
Total	—	62 days	~10 weeks (2 devs)

Issue: Docs promise 3–4 weeks; realistic is 10–12 weeks with 2 engineers

Phase 2 (Months 2–4)

Task	Owner	Duration
Real ELM327 integration	Backend	2 weeks
100 vehicle validation	Data Scientist	6 weeks
PostgreSQL migration	DevOps	2 weeks
Compliance audit	Legal	3 weeks
Monitoring + alerting	DevOps	1 week
Bug fixes from Phase 1	Backend	2 weeks
Total	—	~16 weeks

Phase 3 (Months 4–6)

Task	Owner	Duration
Native vehicle APIs (Tesla/BMW)	Backend	4 weeks
Fleet management dashboard	Frontend	4 weeks
AWS migration	DevOps	2 weeks
Load testing to 50K users	QA	2 weeks
Total	—	~12 weeks

10. FINAL RECOMMENDATION

Technical Feasibility: ✓ YES (7/10)

What Works:

- ✓ OBD-II hardware mature & available
- ✓ Stack (FastAPI, PostgreSQL, Next.js) solid
- ✓ Can scale to 50K+ users
- ✓ MVP shippable in 10–12 weeks (with 2 engineers)

What Needs Work:

- ⚠ Error handling incomplete
- ⚠ Data persistence strategy unclear
- ⚠ Accuracy unvalidated
- ⚠ Compliance/legal missing

Sustainability: ⚠ UNCERTAIN (4/10)

Red Flags:

- ⚫ Zero revenue model (₹52K/month burn)
- ⚫ Legal/compliance not addressed
- ⚫ No accuracy validation plan
- ⚫ Library maintenance risk (pyobd)

Mitigation:

1. ✓ Define freemium/B2B pricing NOW
2. ✓ Legal audit in Phase 2
3. ✓ Validate accuracy on 100 vehicles (Phase 2)
4. ✓ Fork pyobd library (Phase 2)

Recommendation:

✓ **PROCEED WITH MVP** (10–12 week timeline)

Conditions:

1. Resolve 5 critical software gaps (error handling, privacy, liability, logging, mock data)
2. Hire 2 senior engineers (Backend + Frontend)
3. Define revenue model before Beta phase
4. Plan legal audit in parallel (not blocking MVP)

Go/No-Go: GO if you can commit 3 months + 2 FTE engineers + ₹1.5 Lakh budget